

AA 272 Fall 2025. Homework 2

Colin Skinner, colin13@stanford.edu

Instructors: Prof. Grace Gao, Keidai Iiyama, Guillem Casadesus Vila, and Hannah Nabavi

October 3, 2025

Goal of this problem set: In week 2, we learned about the navigation equations and an iterative approach to solving these equations. In this problem set, we will understand why the community uses *weighted* least squares as the standard baseline for GPS and GNSS. To that end, we will investigate least squares practically, theoretically, and geometrically.

Instructions & Notes

1. This problem set is **due at 4:00 PM on October 17, 2025 via Gradescope**.
2. This homework is scored out of **50 points total** and includes a corresponding self-care component. The point value for each homework question is indicated next to the question number. Note that each homework will be equally weighted in the calculation of your final grade.
3. Please note that homework is intended to supplement and reinforce your learning. We design them to guide you through the process of going deeper into the concepts we cover in class, and beyond.
4. You are free to discuss class contents and these problems with others. However, your submitted solutions and code must be your own work, written from scratch.
5. **AI tools** (e.g., Chat GPT, Gemini) can be viewed as search engines or even classmates. They must not be used to directly solve any of the problems. For example, it is acceptable to discuss general topics about GNSS, edit a plot, or how to solve a linear system using `numpy`.
6. **Ensure that you typeset your solutions.** We encourage using the provided notebooks or LaTeX to type up your solutions, but you are free to use Microsoft Word or other editors.
7. Ensure that your solution includes the plots required by the question, **in the specified format** outlined in the problem for full credit. **Please submit your code in the programming assignment entry on Gradescope.** You can submit Colab notebooks and other interactive formats as links in a `.txt` format.
8. If you are spending an excessive amount of time on any part of any question, please post on Ed or come to office hours (see Canvas).
9. Please **start early** so you can address any questions you might have, and have fun!

Problem 1: Solving the Navigation Equations with Newton-Raphson [26 points + 2 bonus]

Goal of this problem: Students will use GNSS measurements collected on an Android phone to perform localization using the Newton-Raphson localization algorithm introduced in Lecture 4.

For this problem, we will use highway data from three different time steps of the Google Smartphone Decimeter Challenge. We have provided three CSV files (`hw2_p1_dataset1_navdata.csv`, `hw2_p1_dataset2_navdata.csv`, `hw2_p1_dataset3_navdata.csv`) on Canvas with the relevant data. We will give you the least squares solution for the first data set (akin to a training data set), but we will not give you the least squares solution for the other data sets.

The columns in the data are as follows:

1. **gps millis:** This is the GPS millis time of the data. Each dataset has a different GPS millis. You do not need to use it for this problem. But, you would need to separate data by gps millis if you were running a whole trajectory (e.g., for your final project).
2. **gnss_id:** This is the GNSS constellation for this data. We will use all constellations in this assignment.
3. **sv_id:** The identifier for the satellite within a constellation. We will use all of the available satellites in this assignment. Satellites are repeated if they transmit in multiple signal bands¹. Still include all entries.
4. **corr prm:** This is the **corrected pseudorange** in meters. Specifically, we have taken the raw pseudorange and removed the satellite clock bias $B^{(k)}$, the inter-signal bias (i.e., inter-constellation) $b_{inter-signal}^{(k)}$, the estimated ionosphere bias $I^{(k)}$, and the estimated troposphere bias $T^{(k)}$. Namely,

$$\rho_{corrected} = \rho_{raw} + B^{(k)} - b_{inter-signal}^{(k)} - I^{(k)} - T^{(k)}$$

Note that compensating for the inter-signal bias is necessary when using multiple constellations. You will learn more about these other biases in the later weeks. But, in effect, these corrections clean the pseudorange measurements. As you will see in problem 2, compensating for these biases is important for solving the navigation equations, especially with least squares.

5. **x_svm:** The satellite x coordinate in ECEF in meters.
6. **y_svm:** The satellite y coordinate in ECEF in meters.
7. **z_svm:** The satellite z coordinate in ECEF in meters.

1.1 [15 points]

Follow the template from Lecture 4 to implement the **Newton-Raphson localization algorithm** for iterative least-squares optimization to solve the navigation equations. Your code should output (1) your optimized position, (2) your optimized clock bias, and (3) the number of iterations until it converges. Check that the optimized position and clock bias for the first data set are (close to) those listed below in ECEF. Also, note the number of iterations until it converged.

$$\mathbf{x} = [-2696209 \text{ m} \quad -4293890 \text{ m} \quad 3856549 \text{ m}]^T$$
$$b_u = 1832 \text{ m}$$

Hint: Below are some suggestions for the code structure. If you prefer object-oriented programming, you may find it easier to pass in the data set as a table-like data structure rather than passing separated arguments.

1. The top-level function (e.g., `newton_raphson(x0, bu_0, max_iterations)`) will take as input an initial guess for position `x0` and clock bias `bu_0`. To help with debugging and to solve the later problems, it will also take the input `max_iterations` which is the maximum number of iterations of Newton-Raphson. This top-level function will return the requisite outputs:
 - (1) your optimized position
 - (2) your optimized clock bias
 - (3) the number of iterations until it converges
2. Implement a function (e.g., `get_geometry_matrix(x_est, x_sv_m, y_sv_m, z_sv_m)`) which takes as input a set of satellite positions (`x_sv_m`, `y_sv_m`, `z_sv_m`) as well as the current position estimate (`x_est`) and outputs the geometry matrix G .
 - For each row of G , the first 3 columns make up a unit vector.
 - You can check the size/shape of G before returning to help debugging.
3. Implement a function (e.g., `get_theoretical_pseudoranges(x_est, b_est, x_sv_m, y_sv_m, z_sv_m)`) which takes as input the current position estimate (`x_est`), the current clock bias estimate (`b_est`), the satellite positions (`x_sv_m`, `y_sv_m`, `z_sv_m`).
 - *Note:* We do **not** need the satellite clock bias since the corrected pseudorange we provide in the data set has already corrected it.
 - This function should output the theoretical pseudorange measurement vector.
 - Again, you can check the size/shape of the theoretical pseudorange measurement vector before returning to help debugging.
4. Either as a function or in the top-level function (i.e., `newton_raphson(x0, bu_0, max_iterations)`), calculate the difference between the theoretical and measured pseudoranges.
5. Implement a function (e.g., `newton_raphson_step(G, delta_rho)`) which takes one step in the Newton-Raphson optimization. That is, it computes the update Δx and Δb_u .
6. Update the current position estimate and clock bias estimate as a function or in the top-level function (i.e., `newton_raphson(x0, bu_0, max_iterations)`).
 - In the top-level function, repeat the iteration until you either:
 1. have updates in Δx and Δb_u that are less than a centimeter, or
 2. hit the maximum number of iterations.

ANSWER

[Replace with your answer]

```
[ ]: import numpy as np
import pandas as pd
from google.colab import drive
import os
from dataclasses import dataclass

if not os.path.ismount("/content/drive"):
    print("Mounting Google Drive...")
    drive.mount("/content/drive")
else:
    print("Google Drive already mounted")
gdrive = True
notebooks_dir = os.path.join("/content/drive/MyDrive/AA 272 - GPS Class/HW2")
```

Mounting Google Drive...
Mounted at /content/drive

```
[ ]: # @title
# Works
def get_geometry_matrix(x_estimate: np.ndarray, x_satellites: np.ndarray):
    """
    Gets geometry matrix G

    x_satellites: Nx3
    x_estimate: 3x1
    """
    n = len(x_satellites)
    dx = x_satellites - x_estimate.T # Nx3 and (3x1).T
    dx_len = np.apply_along_axis(np.linalg.norm, 1, dx).reshape(n, 1)
    dx_hat = dx / dx_len

    ones_column = np.ones((n, 1))

    G_matrix = np.hstack((-dx_hat, ones_column))

    return G_matrix
```

```
[ ]: # @title
"""
Sample data
"""

sat_data = np.array([
    [-22680907.81,      -9478480.541,      -6808698.262],
    [-23096172.86,      -17532377.6,       -5907590.591],
```

```

    [-11993793.74,      -23759441.95,      12948805.17],
    [-16293853.43,      20929977,      33144341.04]
])

```

```

est = np.zeros((3,1))
G_example = get_geometry_matrix(est, sat_data)
print(G_example)

```

```

[[ 0.889192   0.3715984  0.26693111  1.         ]
 [ 0.7804734  0.59245982  0.19963123  1.         ]
 [ 0.40522494  0.8027417  -0.43749116  1.         ]
 [ 0.38382588 -0.49303664 -0.78076409  1.         ]]

```

```

[ ]: def get_theoretical_pseudoranges(x_estimate, x_satellites, b_est):
    """
    Gets theoretical pseudoranges from
    x_satellites: Nx3
    x_estimate: 3x1
    b_est: 1
    """
    n = len(x_satellites)

    def get_single_pseudorange(x_satellite, x_estimate, b_est):
        dist = np.linalg.norm(x_estimate.reshape((1,3)) - x_satellite)
        return dist + b_est

    pseudoranges = np.apply_along_axis(get_single_pseudorange, 1, x_satellites,
    ↪ x_estimate=x_estimate, b_est=b_est)

    return pseudoranges.reshape((n,1))

```

```

[ ]: pseudoranges_example = get_theoretical_pseudoranges(est, sat_data,0)
print(pseudoranges_example)

```

```

[[25507323.34986947]
 [29592517.4947799 ]
 [29597866.90597317]
 [42451159.46349028]]

```

```

[ ]: def newton_raphson_step(G, delta_rho):
    n = len(delta_rho)
    GT_G_inv = np.linalg.inv(G.T @ G)
    delta_rho = delta_rho.reshape(n,1)
    return (GT_G_inv @ G.T @ delta_rho)

```

```
[ ]: step_result = newton_raphson_step(G_example, pseudoranges_example)
print(step_result)
```

```
[[-1.35251782e+08]
 [-2.64721060e+07]
 [ 7.09142722e+07]
 [ 1.36679893e+08]]
```

```
[ ]: def newton_raphson(x_est_0, x_satellites, pseudoranges_measured, bu_est_0,
    ↪max_iterations, show_output):
    """
    1. Guess pos/clock
    2. Calculate G
    3. Get rhos
    4. Calculate d_rho
    5. Get pos/clock updates
    6. Repeat 2-5 until d_rho is less than 1cm
    """

    iteration = 0

    x_estimate = x_est_0.reshape((3,1))
    bias_estimate = bu_est_0

    while(iteration < max_iterations):
        geometry_matrix_G = get_geometry_matrix(x_estimate, x_satellites)
        theoretical_pseudorange = get_theoretical_pseudoranges(x_estimate,
    ↪x_satellites, bias_estimate)

        d_rho = pseudoranges_measured - theoretical_pseudorange

        step_result = newton_raphson_step(geometry_matrix_G, d_rho)
        del_x = step_result[:-1].reshape((3,1))
        del_bias = step_result[-1]
        iteration += 1

        if show_output:
            print(f"Step {iteration} progress: Position estimate={x_estimate.
    ↪flatten()}, Clock bias estimate={bias_estimate}")

        if np.linalg.norm(del_x) < 0.01 and abs(del_bias) < 0.001:
            break
        x_estimate = x_estimate + del_x
        bias_estimate += del_bias

    return x_estimate, bias_estimate[0], iteration
```

```
[ ]: @dataclass
class sat_result:
    pos_ecef: np.ndarray
    clk_bias: float
    iter: int

    pos_lla: np.ndarray = None

def get_ecef_pos(data, initial_guess = None, max_iterations=20, show_output =
↪False):
    satellite_pos = data[['x_sv_m', 'y_sv_m', 'z_sv_m']].to_numpy()
    pseudoranges = data[['corr_pr_m']].to_numpy()

    if initial_guess is None:
        initial_guess = np.array([0,0,0])

    sat_pos_ecef, clk_bias, iter = newton_raphson(initial_guess, satellite_pos,
↪pseudoranges, 0, max_iterations, show_output)

    return sat_result(sat_pos_ecef, clk_bias, iter)
```

```
[ ]: satellite_data_1 = pd.read_csv(f"{notebooks_dir}/data/hw2_p1_dataset1_navdata.
↪csv")
satellite_data_2 = pd.read_csv(f"{notebooks_dir}/data/hw2_p1_dataset2_navdata.
↪csv")
satellite_data_3 = pd.read_csv(f"{notebooks_dir}/data/hw2_p1_dataset3_navdata.
↪csv")
```

```
[ ]: sat_result_1 = get_ecef_pos(satellite_data_1)
sat_result_2 = get_ecef_pos(satellite_data_2)
sat_result_3 = get_ecef_pos(satellite_data_3)
```

1.2 [2 points]

Starting from the Earth center with no clock bias (i.e., $\mathbf{x}_0 = \mathbf{0}$ and $b_{u,0} = 0$), what are your optimized positions and clock biases for the remaining data sets in ECEF? How many iterations were required until it converged?

ANSWER

```
[ ]: """
    =[-2696209m -4293890m 3856549m]
    =1832 m
    """

print(f"Satellite position (m): {sat_result_1.pos_ecef}")
print(f"Satellite clock bias (m): {sat_result_1.clk_bias}")
print(f"Iterations: {sat_result_1.iter}")
```

```
Satellite position (m): [[-2696209.64149143]
[-4293890.83999389]
[ 3856549.78496553]]
Satellite clock bias (m): 1832.2669081674803
Iterations: 5
```

1.3 [3 points]

Using an online converter or GNSS Lib Py, what is your **optimized position in LLA** (for each data set)? Add a screenshot of your position in Google Maps.

```
[ ]: %pip install gnss_lib_py > /dev/null
import gnss_lib_py as glp
```

```
[ ]: sat_result_1.pos_lla = glp.ecef_to_geodetic(sat_result_1.pos_ecef)
sat_result_2.pos_lla = glp.ecef_to_geodetic(sat_result_2.pos_ecef)
sat_result_3.pos_lla = glp.ecef_to_geodetic(sat_result_3.pos_ecef)
```

```
[ ]: print(f"Satellite 1 Result LLA: {sat_result_1.pos_lla.flatten()}")
print(f"Satellite 2 Result LLA: {sat_result_2.pos_lla.flatten()}")
print(f"Satellite 3 Result LLA: {sat_result_3.pos_lla.flatten()}")
```

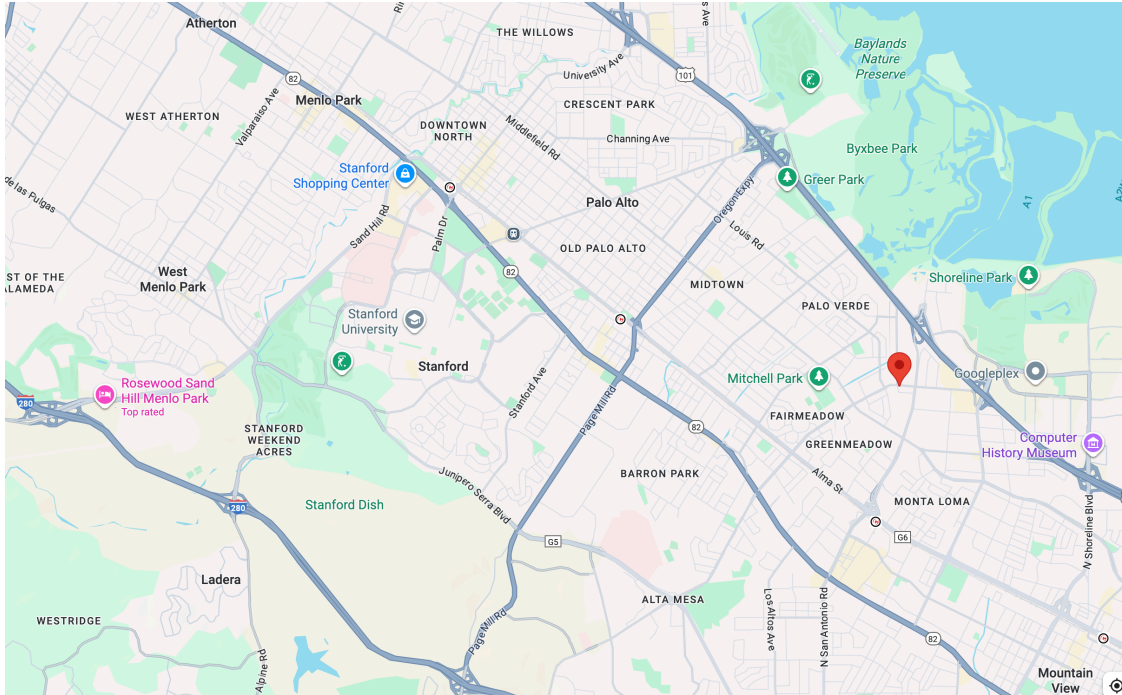
```
Satellite 1 Result LLA: [ 37.44327622 -122.12543509 -29.28286899]
Satellite 2 Result LLA: [ 37.42208887 -122.10385366 -14.08340817]
Satellite 3 Result LLA: [ 37.419674 -122.09300692 -24.15400819]
```

```
[ ]: from IPython.display import Image, display
print(f"Satellite 1 Result LLA: {sat_result_1.pos_lla.flatten()}")
display(Image("/content/drive/MyDrive/AA 272 - GPS Class/HW2/images/dataset_1.
↳png", width=800))
```

Output hidden; open in <https://colab.research.google.com> to view.

```
[ ]: from IPython.display import Image
print(f"Satellite 2 Result LLA: {sat_result_2.pos_lla.flatten()}")
display(Image("/content/drive/MyDrive/AA 272 - GPS Class/HW2/images/dataset_2.
↳png", width=800))
```

```
Satellite 2 Result LLA: [ 37.42208887 -122.10385366 -14.08340817]
```

```
[ ]: from IPython.display import Image
print(f"Satellite 3 Result LLA: {sat_result_3.pos_lla.flatten()}")
display(Image("/content/drive/MyDrive/AA 272 - GPS Class/HW2/images/dataset_3.
↳png", width=800))
```

Output hidden; open in <https://colab.research.google.com> to view.

1.4 [3 points]

Try out different initial positions when initializing the Newton-Raphson algorithm. Use an initial condition on the other side of the planet (i.e., the **antipode** of the user location), and still using $b_{u,0} = 0$. What are your optimized positions and clock biases for the new initial conditions, for each data set? How many iterations were required until it converged?

ANSWER

```
[ ]: sat_result_antipode_1 = get_ecef_pos(satellite_data_1, -sat_result_1.pos_ecef)
sat_result_antipode_2 = get_ecef_pos(satellite_data_2, -sat_result_2.pos_ecef)
sat_result_antipode_3 = get_ecef_pos(satellite_data_3, -sat_result_3.pos_ecef)
```

```
[ ]: print(f"Satellite 1 position (m): {sat_result_1.pos_ecef}")
print(f"Satellite 1 clock bias (m): {sat_result_1.clk_bias}")
print(f"Satellite 1 Iterations: {sat_result_1.iter}")

print(f"Satellite 2 position (m): {sat_result_2.pos_ecef}")
print(f"Satellite 2 clock bias (m): {sat_result_2.clk_bias}")
```

```
print(f"Satellite 2 Iterations: {sat_result_2.iter}")

print(f"Satellite 3 position (m): {sat_result_3.pos_ecef}")
print(f"Satellite 3 clock bias (m): {sat_result_3.clk_bias}")
print(f"Satellite 3 Iterations: {sat_result_3.iter}")
```

```
Satellite 1 position (m): [[-2696209.64149143]
 [-4293890.83999389]
 [ 3856549.78496553]]
Satellite 1 clock bias (m): 1832.2669081674803
Satellite 1 Iterations: 5
Satellite 2 position (m): [[-2695358.10787255]
 [-4296127.07337435]
 [ 3854691.78389111]]
Satellite 2 clock bias (m): 613.3500210501871
Satellite 2 Iterations: 5
Satellite 3 position (m): [[-2694627.03315257]
 [-4296768.45811226]
 [ 3854472.80878812]]
Satellite 3 clock bias (m): -272.4524545987185
Satellite 3 Iterations: 5
```

```
[ ]: print(f"Satellite 1 Result LLA: {sat_result_1.pos_lla.flatten()}")
      print(f"Satellite 2 Result LLA: {sat_result_2.pos_lla.flatten()}")
      print(f"Satellite 3 Result LLA: {sat_result_3.pos_lla.flatten()}")
```

```
Satellite 1 Result LLA: [ 37.44327622 -122.12543509 -29.28286899]
Satellite 2 Result LLA: [ 37.42208887 -122.10385366 -14.08340817]
Satellite 3 Result LLA: [ 37.419674 -122.09300692 -24.15400819]
```

It required the same number of iterations, and it still arrived at the same answer.

1.5 [3 points]

Provide one example of an initial position (still using $b_{u,0} = 0$) where Newton-Raphson **does not converge in 20 iterations**. What is the estimated position and clock bias when it hits 20 iterations? Repeat for all three data sets.

Hint: You may need to try a point that is a few Earth radii away.

ANSWER

```
[ ]: sat_result_super_far_1 = get_ecef_pos(satellite_data_1, 6.002*sat_result_1.
      ↪pos_ecef)
      sat_result_super_far_2 = get_ecef_pos(satellite_data_2, -6.3*sat_result_1.
      ↪pos_ecef)
      sat_result_super_far_3 = get_ecef_pos(satellite_data_3, 6.006*sat_result_1.
      ↪pos_ecef)
```

```
[ ]: sat_result_super_far_1.pos_lla = glp.ecef_to_geodetic(sat_result_super_far_1.
    ↪pos_ecef)
sat_result_super_far_2.pos_lla = glp.ecef_to_geodetic(sat_result_super_far_2.
    ↪pos_ecef)
sat_result_super_far_3.pos_lla = glp.ecef_to_geodetic(sat_result_super_far_3.
    ↪pos_ecef)

[ ]: print(f"Satellite 1 position (m): {sat_result_super_far_1.pos_ecef.flatten()}")
print(f"Satellite 2 position (m): {sat_result_super_far_2.pos_ecef.flatten()}")
print(f"Satellite 3 position (m): {sat_result_super_far_3.pos_ecef.flatten()}")
print()
print(f"Satellite 1 position LLA (m): {sat_result_super_far_1.pos_lla.
    ↪flatten()}")
print(f"Satellite 2 position LLA (m): {sat_result_super_far_2.pos_lla.
    ↪flatten()}")
print(f"Satellite 3 position LLA (m): {sat_result_super_far_3.pos_lla.
    ↪flatten()}")
```

```
Satellite 1 position (m): [-1.28721148e+13 -2.36744672e+13  1.75581033e+13]
Satellite 2 position (m): [-2.31709241e+13 -4.20501412e+13  3.32938908e+13]
Satellite 3 position (m): [ 1.44189967e+12  1.66298660e+12 -4.07030339e+12]
```

```
Satellite 1 position LLA (m): [ 3.30868690e+01 -1.18533499e+02  3.21629961e+13]
Satellite 2 position LLA (m): [ 3.47396170e+01 -1.18856119e+02  5.84259234e+13]
Satellite 3 position LLA (m): [-6.15973843e+01  4.90729362e+01  4.62730050e+12]
```

The position is extremely far away, and the altitude is on the order of $10^{13}m$.

Bonus 1 [1 point]

The actual ground truth point for the first dataset is

Latitude: 37.44329905 degrees

Longitude: -122.12575639 degrees

This ground truth is a post-processed localization using a high-accuracy, high-precision GNSS receiver mounted on the vehicle. Use the Lecture 3 slides to estimate the **shift** between the least squares solution from Newton-Raphson to the ground truth. Report the value in degrees and in meters north (latitude) and east (longitude) from the least squares solution to the ground truth.

Hint: You can cross-check your solution with an online calculator (e.g., from Meridian Outpost)

ANSWER

```
[37]: ground_truth = np.array([37.44329905, -122.12575639])
newton_raphson_values = np.array([37.42208887, -122.1038536])

difference_deg = ground_truth - newton_raphson_values
difference_m = np.array([difference_deg[0], difference_deg[1] * np.
    ↪cos(ground_truth[0] * np.pi / 180.0)]) * 111320
```

```
print(f"Difference (°): {difference_deg[0]:.5f}° N, {-difference_deg[1]:.5f}° W")
print(f"Difference (m): {difference_m[0]:.5f}m N, {-difference_m[1]:.5f}m W")
```

Difference (°): 0.02121° N, 0.02190° W

Difference (m): 2361.11724m N, 1935.83679m W

Bonus 2 [1 point]

This shift between the least squares and the actual ground truth is consistent across all three data sets. What is causing this shift? How can we address this shift?

Hint: How long does it take for the satellite signal to reach us?

Hint: You do not need to implement your solution to address the shift to get credit, but you may find it useful to implement your solution to check your understanding.

ANSWER

Since the satellites themselves are moving, their positions at the time of reception is different than their position at the time they send out their transmission. That is, when the module receives the satellite data, it checks its almanac to figure out where the satellites are *at the time of reception*; however, when the satellites sent the signal, they were in different positions so you need to propagate backward to find their *true* position when they transmit their messages

We could address this in the following manner: - Obtain pseudoranges for the satellites - Approximate how far away the satellite is - Use the almanac to propagate backward to figure out where the satellites are at the times of their transmission - Recalculate true position

Problem 2 (10 points): Theoretical Motivation for WLS

Goal of this problem: Students will learn about why we often use the *weighted* least-squares solution rather than the (ordinary) least-squares solution of the navigation equations.

What are the assumptions of this solution? Are these assumptions valid?

In Lecture 4, we derived the (ordinary) least-squares (OLS) solution to the linearized navigation equations as:

$$\delta \mathbf{y} = \begin{bmatrix} \delta \mathbf{x} \\ \delta b \end{bmatrix} = (G^\top G)^{-1} G^\top \delta \rho,$$

where $\delta \mathbf{y} \in \mathbb{R}^4$ is the correction to our state consisting of a position correction $\delta \mathbf{x} \in \mathbb{R}^3$ and a clock bias correction $\delta b \in \mathbb{R}$, $\delta \rho \in \mathbb{R}^N$ is the vector of pseudorange residuals for N visible satellites, and $G \in \mathbb{R}^{N \times 4}$ is the geometry matrix.

Ideally, we want the pseudorange residuals to be near zero (i.e., $\delta \rho \approx \mathbf{0}$), but we cannot achieve zero residuals in practice due to measurement errors. For example, the residuals can be 10 – 100 m in harsh urban environments!

In what sense is the (ordinary) least-squares solution (OLS) the *best* or *optimal* solution?

Building on the shoulders of giants, the work of Gauss (1777–1855) and Markov (1856–1922) proved that the OLS estimator is the *best linear unbiased estimator (BLUE)* under certain assumptions on the error.

First, what does BLUE mean?

- *Best* refers to the estimator with the minimum variance among all possible estimators with the restrictions and assumptions below.
- *Linear* refers to restricting to a linear model (i.e., $\delta \rho = G \delta \mathbf{y}$ is linear).
- *Unbiased* [1] refers to restricting the estimator to be centered around the true value if you repeat the estimation many times with new data.
- *Estimator* refers to the procedure to *estimate* an unknown quantity from data.

The assumptions on the error are quite important.

Specifically, this is the error ϵ in the linear model $\delta \rho = G \delta \mathbf{y} + \epsilon$.

The three assumptions [2] are below. The figure below illustrates these assumptions more graphically.

1. **Errors are uncorrelated.** The error $\epsilon^{(1)}$ from the pseudorange measurement of satellite 1 is not correlated with the error $\epsilon^{(2)}$ from the pseudorange measurement of satellite 2, and so on with the other satellites.
2. **Errors have equal variance** (AKA “spherical errors”). The errors $\epsilon^{(1)}$ from the pseudorange measurement in satellite 1 have a distribution with the same variances as the errors $\epsilon^{(2)}$ in satellite 2, and so on with the other satellites.

3. **Errors have an expected value (i.e., mean) of zero.** If we get many measurements from satellite 1, then the mean of the pseudorange residuals will be zero.

In practice, the second assumption is almost never true. Fortunately, Aitken (1895–1967) found a way to generalize least-squares estimation to remove the first and second assumptions. Aitken’s approach is called *generalized least-squares (GLS)* [3], but it requires good knowledge of the error distribution, which is quite difficult to achieve in practice. Moreover, approaches to estimate the correlation and variance between errors tend to be quite computationally heavy.

So, instead, we only drop the second assumption and try to account for the differences in variance between measurements, but we do not account for the correlation between errors. This approach is called **weighted least-squares (WLS)**. It turns out to be pretty good, which is why people still use it. We will see more about WLS in Lecture 6.

To provide intuition, we want to put a *larger* weight on satellites where the variance is *small* since these measurements provide a more precise understanding of the user location. Likewise, we want to put a *smaller* weight on satellites where the variance is *large* (Figure above).

To incorporate relative weighting of the pseudorange measurements, we will redefine our problem statement for the “best” $\delta\mathbf{y}$ solution to be:

$$\arg \min_{\delta\mathbf{y}} \|\delta\rho - G \delta\mathbf{y}\|_W^2,$$

where $W = \text{diag}(w_1, w_2, \dots, w_N)$ is a diagonal weighting matrix with positive weights w_i , and $\|\cdot\|_W$ is the Mahalanobis norm, defined as:

$$\|\mathbf{z}\|_W^2 = \mathbf{z}^\top W \mathbf{z},$$

where $\mathbf{z} \in \mathbb{R}^N$.

To provide intuition, the normal Euclidean distance (i.e., the Euclidean norm) arises if $W = I$, where I is the identity matrix. The contours of the Euclidean norm are circles whereas the contours of the Mahalanobis norm are ellipses.

[1]: The restriction on “unbiased” estimation is critical. There are biased estimators with lower variance. For example, “regularization” is a common bias that is introduced to lower variance, especially in a least-squares context. For more, I recommend AA 222 (Optimization) and CS/STATS 229 (Machine Learning).

[2]: Often, people incorrectly list an assumption or requirement that the errors *must* follow a multivariate Gaussian (i.e., a normal distribution). However, we do not *require* that the errors are distributed as a multivariate Gaussian. But, it turns out that we can make even stronger claims about least-squares estimation if the errors are Gaussian, as we assumed in Lecture 4. In many cases, GPS/GNSS errors can be well-modelled as Gaussians. For more, I recommend AA 273 (State Estimation) and CS/STATS 229 (Machine Learning).

[3]: Check out Chapter 6 of the GPS textbook and [Wikipedia: Generalized least squares](#).

```
[ ]: import numpy as np
import pandas as pd
```

```

from google.colab import drive
import os
from dataclasses import dataclass

if not os.path.ismount("/content/drive"):
    print("Mounting Google Drive...")
    drive.mount("/content/drive")
else:
    print("Google Drive already mounted")
gdrive = True
notebooks_dir = os.path.join("/content/drive/MyDrive/AA 272 - GPS Class/HW2")

```

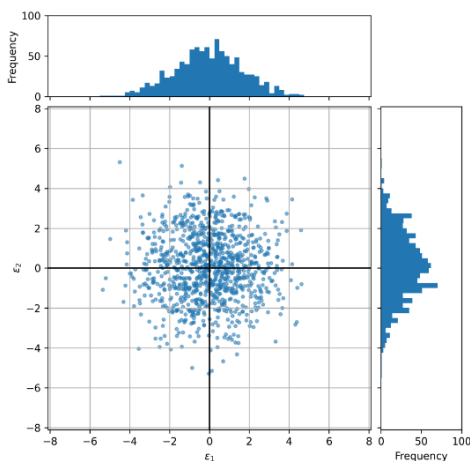
Mounting Google Drive..
Mounted at /content/drive

```

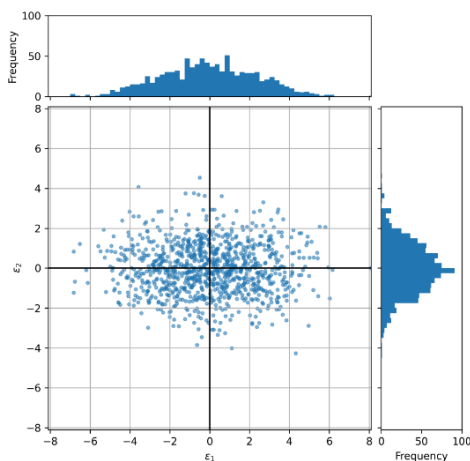
[ ]: # @title
from IPython.display import Image, display

display(Image(f"{notebooks_dir}/images/Correlations.png", width=800))

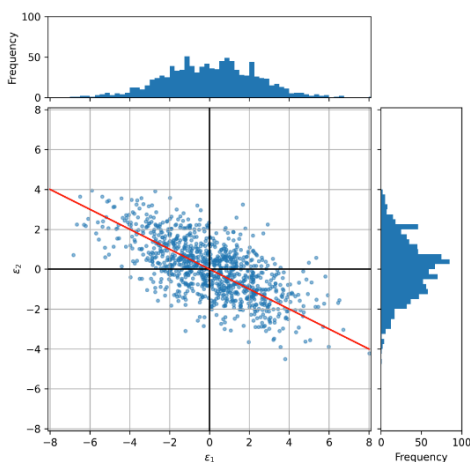
```



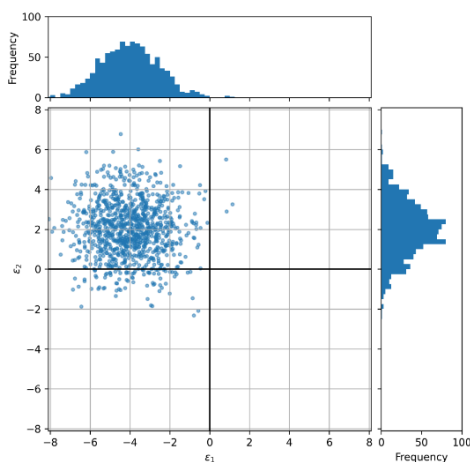
(a) Uncorrelated, spherical, zero-mean errors. Ideal for OLS.



(b) Uncorrelated, **non-spherical**, zero-mean errors. Solvable with WLS.



(c) **Correlated** (red line), **non-spherical**, zero-mean errors. Solvable with GLS.



(d) Uncorrelated, spherical, **non-zero-mean** errors. Try to compensate for bias before OLS.

2.1 [4 Points]

By following very similar steps as in Lecture 4, show that the solution to the WLS problem is:

$$\delta \mathbf{y} = (\mathbf{G}^\top \mathbf{W} \mathbf{G})^{-1} \mathbf{G}^\top \mathbf{W} \delta \boldsymbol{\rho}$$

This is called the **weighted least-squares** solution.

ANSWER

```
[ ]: display(Image(f"{notebooks_dir}/images/Derivation.jpg", width=800))
```


$$\begin{aligned}
& \frac{\partial}{\partial \delta y} \min_{\delta y} \| \delta p - G \delta y \|_W^2 \quad \|z\|_W^2 = z^T W z \\
&= \frac{\partial}{\partial \delta y} (\delta p - G \delta y)^T W (\delta p - G \delta y) \\
&= \frac{\partial}{\partial \delta y} (\delta p^T - \delta y^T G^T) W (\delta p - G \delta y) \\
&= \frac{\partial}{\partial \delta y} [\delta p^T W \delta p - \delta y^T \underbrace{G^T W}_{\text{diagonal}} \delta p - \delta p^T W G \delta y + \delta y^T G^T W G \delta y] \\
&= 0 - G^T W \delta p - \delta p^T W G + 2 G^T W G \delta y \\
&= -G^T W \delta p - G^T W \delta p + 2 G^T W G \delta y \\
&= -2 G^T W \delta p + 2 G^T W G \delta y \\
&G^T W \delta p = (G^T W G) \delta y \\
&\boxed{\delta y = (G^T W G)^{-1} G^T W \delta p}
\end{aligned}$$

The derivation was carried out with the weighting matrix in the middle of the terms. Note that in the sixth line, when taking the transpose of the second term, W^T is allowed to be reduced to W because it is a diagonal matrix.

2.2 [2 Points]

Mathematically, why is it important for the diagonal weights of $\mathbf{W}$ to be **positive** for our defined problem statement and derivation?

Hint:

If you are stuck, head to Wolfram Alpha[4] and compare the output plots for each:

- Both positive weights: $\{x, y\} * \{\{1/2, 0\}, \{0, 1\}\} * \{x, y\}$
- One negative weight: $\{x, y\} * \{\{1/2, 0\}, \{0, -1\}\} * \{x, y\}$
- Different negative weight: $\{x, y\} * \{\{-1/2, 0\}, \{0, 1\}\} * \{x, y\}$
- Both negative weights: $\{x, y\} * \{\{-1/2, 0\}, \{0, -1\}\} * \{x, y\}$

[4]: Available at this link: <https://www.wolframalpha.com/>

ANSWER

With regards to the definition:

For the least-squares problem to be well-posed and for the Newton Raphson method to converge to a minimum, W must be strictly convex. This convexity is guaranteed if and only if W is positive definite, meaning that it has a minimum point (and we want to minimize errors).

Similar to how multiplying a positive number by a negative number is negative, when you multiply this W matrix in the pseudo-inverse equation, if it is not positive definite, then it will not converge.

Plots

The Wolfram alpha plot for positive weights is concave up in both the X and Y directions with a global minimum. For one negative weight, the plot creates a saddle point (no global max or min). For both negative weights, the plot is completely concave down, with a global maximum.

Intuitively, because the Newton-Raphson method needs to find the minimum to settle on a final value for position, the gradient descent method of “subtracting” the error will put the current estimate closer to the final minimum-error value.

However, for negative weights on any diagonal entry, traveling down the gradient will not allow it to converge because it is concave down. This is equivalent to instead **adding** error, which causes the result to diverge.

e.g. If you are trying to converge at (1,1), and your current position is at (2,2), rather than subtracting your error of (1,1), it will add to give you a new estimate of (3,3).

2.3 [2 Points]

Why would some pseudorange measurements have better quality than others? In other words, why is assumption 2 (Errors have equal variance/spherical errors) violated in real measurements? Provide at least two concrete examples.

Hint: There are *many* possible answers. This problem set and lectures 1, 5, and 6 may provide some insights, but feel free to think outside the box!

ANSWER

Atmosphere and physical obstructions

Satellites that are at low elevations travel through more of the atmosphere, and thus receive more attenuation. They are also more likely to go through/around trees, mountains, and buildings, which further attenuates the signal. Therefore, error in the direction of that satellite will have more variance than satellites that are overhead.

Satellites in View (SIV)

Having more satellites in view increases the likelihood of having 4 really strong satellite signals. On the other hand, if there are fewer satellites in view, and if one or more of them are not strong signals, then the pseudorange measurements are more susceptible to errors.

Outdated ephemeris data

Having outdated ephemeris data causes the receiver to have incorrect satellite locations.

2.4 [2 Points]

Provide at least one concrete example where the errors are **correlated** (i.e., violating assumption 1), and at least one concrete example where the errors do not have **zero means** (i.e., violating assumption 3). *Note that weighted least-squares cannot handle these errors on its own.

Note: You may consider the corrections we applied in problem 1 and other insights from the problem 2 and 3 descriptions. However, there are *many* possible answers. Lectures 1, 5, and 6 may provide some insights, but feel free to think outside the box!

Hint: If you are stuck, think back to problem 2 on homework 1. From the skyplots, there are often satellites with similar elevation and azimuth in a multi-constellation setting. Are there errors that similarly affect satellites in nearby elevation and azimuth?

ANSWER

Correlated Errors

Having more noise—like thermal noise, for example—will cause the noise floor of the signal to increase. Because the GPS signals are already very faint, having more noise on top of it will increase the likelihood that the auto-correlation will not produce a valid Gold code for a satellite. Because this is noise, it will typically have zero-mean error. There could also be noise on the end of the receiver, which would affect its measurements for all satellites.

Non-zero-mean Error

If the ephemeris is outdated, then the positions of all satellites will be incorrect. This causes there to be a constant non-zero-mean error for each satellite.

Problem 3: Satellite Geometry [14 points + 3 bonus]

Goal of this problem: Students will learn about Dilution of Precision (DOP) in nominal and adverse settings.

Recall from Lecture 4 that $(G^\top G)^{-1}$ tells us how the user-satellite geometry affects our Dilution of Precision and state uncertainty, where G is the geometry matrix.

In class, we created the G matrix with unit direction vectors in the ECEF (Earth-Centered-Earth-Fixed) reference frame; however, it is often more intuitive and practical to understand our state uncertainty in our local reference frame, i.e., ENU frame (East-North-Up).

Thus, let us define a new geometry matrix $\tilde{G}$ as:

$$\tilde{G} = \begin{bmatrix} \left(-\mathbf{1}_{\text{ENU}}^{(1)}\right)^\top & 1 \\ \left(-\mathbf{1}_{\text{ENU}}^{(2)}\right)^\top & 1 \\ \vdots & \vdots \\ \left(-\mathbf{1}_{\text{ENU}}^{(N)}\right)^\top & 1 \end{bmatrix}$$

Notice, this matrix is almost identical to the one we derived in class, but where we use a unit direction vector that is defined in our local, ENU reference frame (denoted by $\mathbf{1}_{\text{ENU}}^{(1)}$).

Let us define our local DOP matrix as:

$$\tilde{H} = (\tilde{G}^\top \tilde{G})^{-1},$$

which corresponds to the following $\mathbb{R}^{4 \times 4}$ matrix:

$$\tilde{H} = \begin{bmatrix} \text{EDOP}^2 & \cdot & \cdot & \cdot \\ \cdot & \text{NDOP}^2 & \cdot & \cdot \\ \cdot & \cdot & \text{VDOP}^2 & \cdot \\ \cdot & \cdot & \cdot & \text{TDOP}^2 \end{bmatrix},$$

where EDOP is our East DOP, NDOP is our North DOP, VDOP is our Vertical DOP (i.e., up direction), and TDOP is the same as defined in lecture 4, i.e., our Time DOP.

Each of these terms refers to how much the satellite geometry *dilutes* (or degrades) our precision in the user position and time when the DOP value > 1 or improves our precision when the DOP value < 1 .

For example, a high EDOP and low NDOP mean that we have worse precision in the East direction and better precision in the North direction.

The off-diagonal terms are generally not zero, but will generally be smaller in magnitude¹.

With this matrix $\tilde{H}$, we additionally get the following definitions of Horizontal DOP (HDOP) and Position DOP (PDOP):

$$\text{HDOP} = \sqrt{\text{EDOP}^2 + \text{NDOP}^2}, \text{PDOP} = \sqrt{\text{EDOP}^2 + \text{NDOP}^2 + \text{VDOP}^2}.$$

```
[1]: import numpy as np
import pandas as pd
from google.colab import drive
import os
from dataclasses import dataclass

if not os.path.ismount("/content/drive"):
    print("Mounting Google Drive...")
    drive.mount("/content/drive")
else:
    print("Google Drive already mounted")
gdrive = True
notebooks_dir = os.path.join("/content/drive/MyDrive/AA 272 - GPS Class/HW2")
```

Mounting Google Drive...
Mounted at /content/drive

3.1 [2 points]

Create a function called `get_unit_vector(az, el)`, which takes in a satellite's azimuth and elevation angle in degrees, `az` and `el`, respectively, and outputs the 3-dimensional unit vector which represents the “look” direction to that satellite in the local, ENU reference frame.

If you are using GNSS Lib Py, the function can be `get_unit_vector(navdata)`.

Hint: This [reference document](#) or chapter 6.1.2 of the GPS textbook might help in defining the unit direction vector in the local frame, given an azimuth and elevation angle.

Take a look at the section on *local horizon coordinates* in the linked file.

Hint: A unit vector (also, the output of your function!) has magnitude 1.

ANSWER

```
[2]: from numpy.linalg import norm
import numpy as np
```

```
[3]: RAD_TO_DEG = 180.0/np.pi
DEG_TO_RAD = np.pi/180.0
```

```
[4]: # I used my naming convention but wrapped it
def topo_horizon_to_ENU(azimuth, elevation, use_degree=False):
    azimuth = azimuth * DEG_TO_RAD if use_degree else azimuth
    elevation = elevation * DEG_TO_RAD if use_degree else elevation

    east = np.sin(azimuth) * np.cos(elevation)
    north = np.cos(azimuth) * np.cos(elevation)
    up = np.sin(elevation)

    r_enu = np.array([east, north, up])
```

```

# Should be unit vector, but normalizing just in case
r_norm = norm(r_enu)

r_enu = r_enu / r_norm

return r_enu

def get_unit_vector(az, el, use_degree = True):
    return topo_horizon_to_ENU(az, el, use_degree)

```

Using the skyplots from homework 1, recall that the GPS satellites can only appear in the blue subset of the skyplot in the figure below.

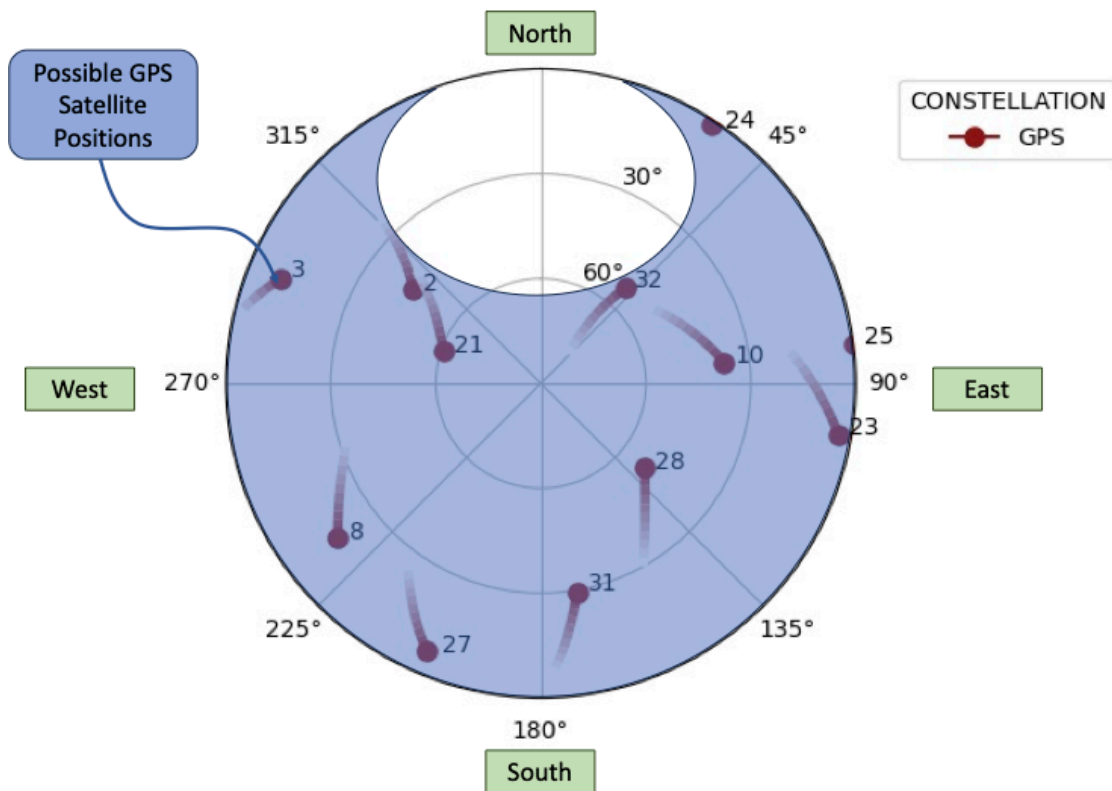
These satellite locations, along with their elevation and azimuth are included on Canvas as a CSV file named `hw2_p3_az_el_data.csv`

```

[27]: from IPython.display import Image, display

display(Image(f"{notebooks_dir}/images/HW2_P4_DOP_OpenSky.png", width=600))

```



3.2 [1 point]

Using your function, `get_unit_vector`, what is the ENU direction vector to **GPS satellite 27**? (Azimuth: 203.0°, Elevation: 6.7°) You can intuitively check your answer by referencing the skyplot.

ANSWER

```
[6]: satellite_data = pd.read_csv(f"{notebooks_dir}/data/hw2_p3_az_el_data.csv")
      satellite_27 = satellite_data[satellite_data['sv_id'] == 27]
```

```
[7]: azimuth = satellite_27['az_sv_deg'].iloc[0]
      elevation = satellite_27['el_sv_deg'].iloc[0]

      print(f"Azimuth: {float(azimuth)}deg")
      print(f"Elevation: {float(elevation)}deg")

      print(f"ENU direction vector: {get_unit_vector(azimuth, elevation)}")
```

```
Azimuth: 203.0083505975292deg
Elevation: 6.707822482097567deg
ENU direction vector: [-0.3881897  -0.91414716  0.11680633]
```

[Replace with your answer]

3.3 [2 points]

Create a function called `compute_dop_matrix(az_arr, el_arr)` (or if using GNSS Lib Py, `compute_dop_matrix(navdata)`) which takes in an array of satellite azimuth and elevation angles in degrees, `az_arr` and `el_arr`, respectively, and calls your `get_unit_vector` function from part (3.1) to output the DOP matrix $\tilde{\mathbf{H}}$ in the ENU frame.

Plot the resulting DOP matrix for the open sky condition as a heatmap.

Hint: Use the following function for heatmap generation.

```
[8]: import matplotlib.pyplot as plt

def plot_dop_matrix(H, cmap='RdBu_r', vmin=-20, vmax=20,
                    symmetric_color=True, label_nums=True):
    if symmetric_color:
        if (vmin is not None) or (vmax is not None):
            if not np.isclose(np.abs(vmin), np.abs(vmax)):
                raise ValueError(f"Cannot force symmetric color since vmin_
↪({vmin}) is not of equal magnitude to vmax ({vmax})")
            else:
                # Estimate the vmin and vmax from values
                vmax = np.max(np.abs(H))
                vmin = -vmax

        # Plot the matrix as an image
        plt.imshow(H, cmap=cmap, vmin=vmin, vmax=vmax)
```

```

plt.colorbar()

# Set the tick marks
ticks = [0, 1, 2, 3]
tick_labels = ["East", "North", "Up", "Time"]
plt.xticks(ticks, tick_labels)
plt.yticks(ticks, tick_labels)

# Label the numbers
if label_nums:
    for (j,i), label in np.ndenumerate(H):
        plt.text(i,j, f"{label:.3f}", ha='center', va='center')

```

ANSWER

```

[9]: def compute_dop_matrix(az_arr, el_arr, use_degree=True):
      """Both arrays are python lists or numpy vectors (1D)"""

      num_satellites = len(az_arr)

      G = np.ones((num_satellites, 4))
      for i, (az, el) in enumerate(zip(az_arr, el_arr)):
          enu_vector = get_unit_vector(az, el, use_degree)

          G[i, 0:3] = - enu_vector.reshape((1,3))

      return np.linalg.inv(G.T @ G)

```

```

[10]: azimuths = satellite_data['az_sv_deg']
      elevations = satellite_data['el_sv_deg']

      H = compute_dop_matrix(azimuths, elevations, use_degree=True)

      print(f"H matrix:\n{H}")

```

```

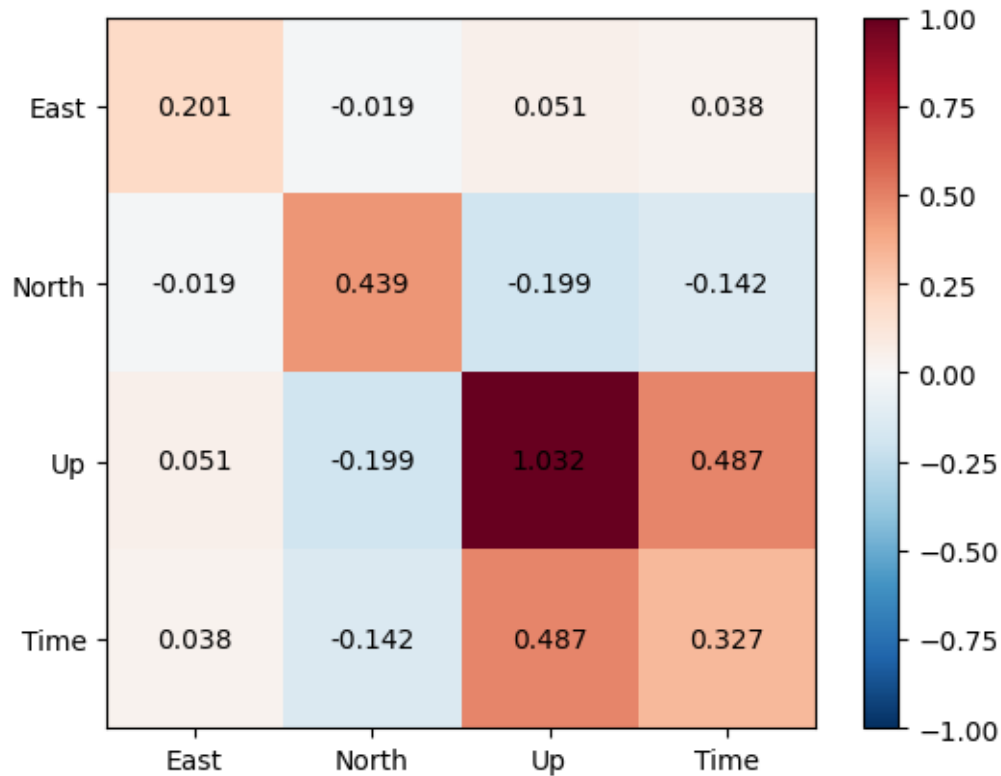
H matrix:
[[ 0.20097502 -0.01913473  0.05142934  0.03780211]
 [-0.01913473  0.43919681 -0.19899284 -0.14190813]
 [ 0.05142934 -0.19899284  1.03224535  0.48714849]
 [ 0.03780211 -0.14190813  0.48714849  0.32733601]]

```

```

[11]: plot_dop_matrix(H, vmin=-1, vmax=1)

```

3.4 [2 points]

Create a function called `compute_pdop(H)`, which uses the output of your `compute_dop_matrix` function and computes the **PDOP** for the provided satellites. Check that the PDOP of the provided satellites is **1.29**.

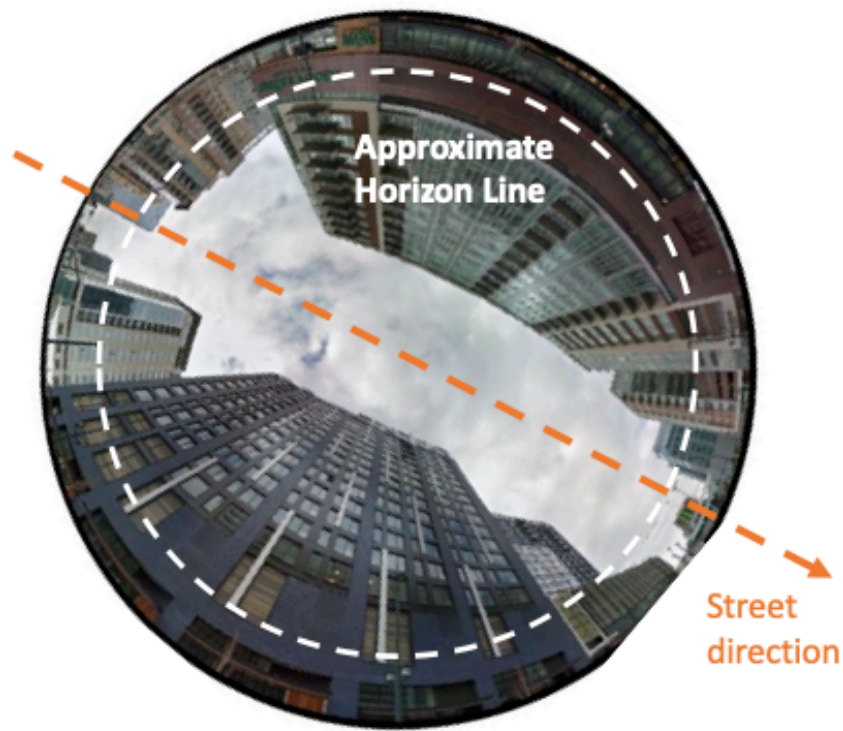
ANSWER

```
[12]: def compute_pdop(H):
      return np.sqrt(H[0,0] + H[1,1] + H[2,2])
```

Now imagine that there were buildings around the receiver, forming an urban canyon, as shown below

```
[13]: from IPython.display import Image, display

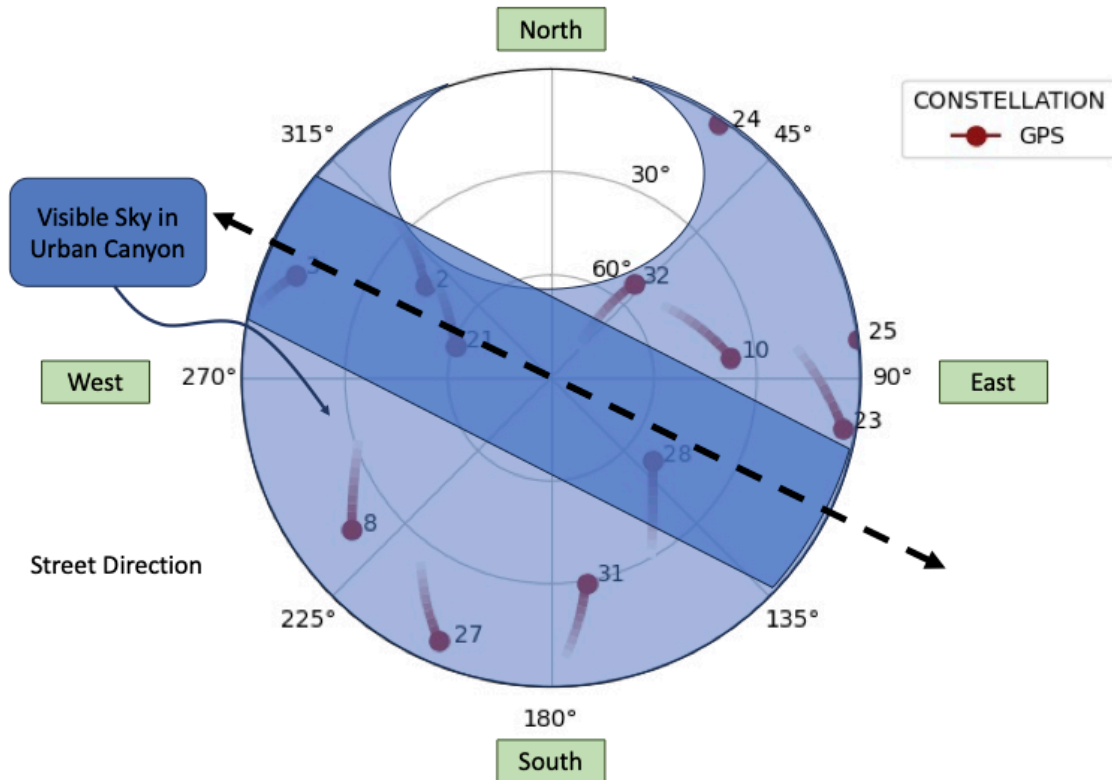
display(Image(f"{notebooks_dir}/images/SalesforceDataImageAA272.png",
             width=400))
```



These buildings will block the satellite signals. If a signal does reach the receiver, it will have bounced on other buildings before reaching the receiver. So, the measured pseudorange will have large errors (The technical term is **non-line-of-sight bias** or **NLOS bias**) Suppose we can correctly remove all the signals that are blocked or degraded from the buildings. The only remaining visible GPS satellites are 2, 3, 21, 28 (left to right) in the figure below.

```
[14]: from IPython.display import Image, display

display(Image(f"{notebooks_dir}/images/HW2_P4_DOP_UrbanCanyon.png", width=600))
```



```
[15]: satellites_in_view = satellite_data[satellite_data['sv_id'].isin([2,3,21,28])]

azimuths = satellites_in_view['az_sv_deg']
elevations = satellites_in_view['el_sv_deg']

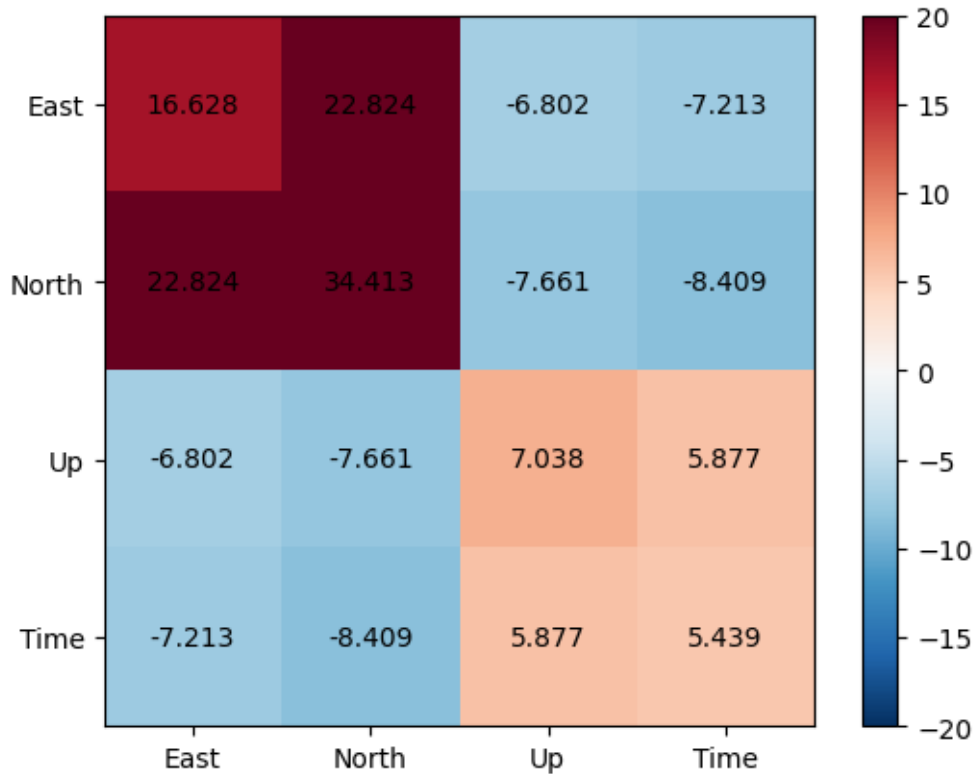
H = compute_dop_matrix(azimuths, elevations, use_degree=True)

print(f"H matrix:\n{H}")

plot_dop_matrix(H, vmin=-20, vmax=20)
```

H matrix:

```
[[16.6282074  22.82428202 -6.8016688  -7.21293016]
 [22.82428202  34.41260325 -7.66052989 -8.40886355]
 [-6.8016688  -7.66052989  7.03771467  5.87671075]
 [-7.21293016 -8.40886355  5.87671075  5.43863782]]
```



The North/South dilution of precision is very high, and the East/West is fairly high as well.

3.5 [4 points]

Using your function, `compute_dop_matrix(az_arr, el_arr)`, output the **new DOP matrix** with only the four available satellites visible from within the urban canyon. Briefly comment on the difference you see in the DOP matrix. Specifically, what has changed in the diagonal components (i.e., EDOP, NDOP, VDOP, and TDOP) and the off-diagonal components?

ANSWER

[Replace with your answer]

Deliverable: The new DOP matrix $\tilde{\mathbf{H}}$ (as a heatmap plot) and brief (roughly 1-2 sentences) comments to the questions.

3.6 [3 points]

Using your function, `compute_pdop(H)`, what is the **PDOP** within the urban canyon. Is it higher or lower than before? Why?

ANSWER

```
[16]: print(f"Urban Canyon PDOP: {compute_pdop(H)}")
```

Urban Canyon PDOP: 7.620926801987277

As explained in 3.5, the East/West and North/South dilutions of precision are very large compared to the full-sky view. This is likely due to urban canyon being fairly small and only in a thin line. Thus, the data is not as precise as it would be with a wide variety of locations in the sky.

There is an ongoing effort to develop a receiver that can track weak GPS signals near the Moon (Example: LuGRE Mission). Figure below shows the skyplot of the GPS satellites at different locations on the Moon, where the GPS satellite directions are clustered in the lunar user's sky. In this bonus problem, we will think about the challenges of using GPS signals on the Moon by computing the DOP values.

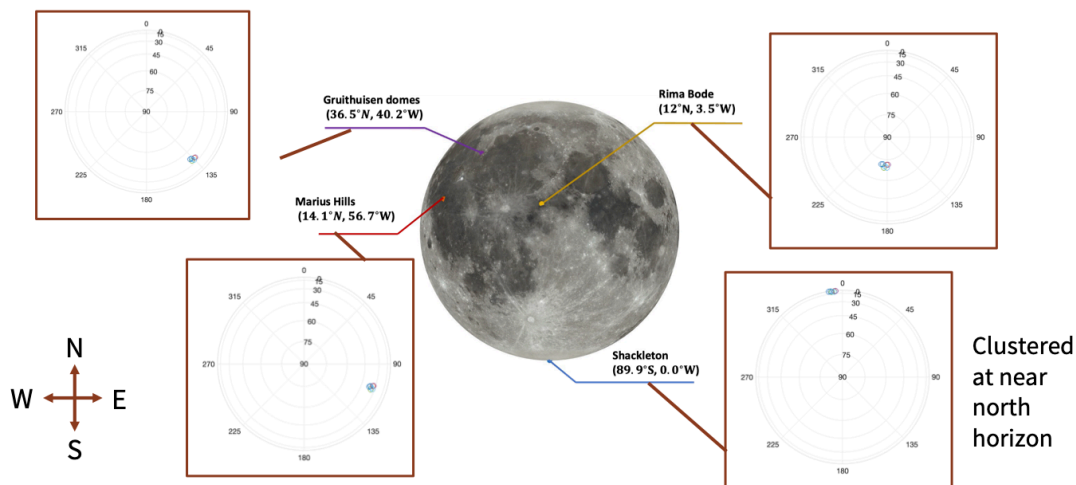
We are provided with four CSV files on Canvas for the following lunar surface locations:

1. GruithuisenDomes: hw2_p3_bonus_Shackleton.csv
2. MariusHills: hw2_p3_bonus_MariusHills.csv
3. RimaBode: hw2_p3_bonus_RimaBode.csv
4. Shackleton: hw2_p3_bonus_GruithuisenDomes.csv

Each file contains the azimuth and elevation of the GPS satellites that are visible from the 4 locations on the lunar surface shown in the figure below (These values were computed using the GPS satellite positions and lunar position on August 1st, 2022)

```
[17]: from IPython.display import Image, display

display(Image(f"{notebooks_dir}/images/LunarSkyPlots.png", width=800))
```



Hint: The column labels of the azimuth and elevation are the same as the data we provided for Problem 3. If you are using gnsslibpy, you can load the csv files as `navdata` using the following function. Then, you can use the same function you implemented in Problem 3 to compute the DOP matrices.

```
import gnss_lib_py as glp
```

```

data_sh = glp.NavData(csv_path="hw2_p3_bonus_Shackleton.csv")
data_mh = glp.NavData(csv_path="hw2_p3_bonus_MariusHills.csv")
data_rb = glp.NavData(csv_path="hw2_p3_bonus_RimaBode.csv")
data_gd = glp.NavData(csv_path="hw2_p3_bonus_GruithuisenDomes.csv")

```

3.7 Bonus [1 point]

Using your function, `compute_dop_matrix(az_arr, el_arr)` and `compute_pdop(H)`, output the DOP matrix and PDOP value for each of the surface locations. At which surface location do you observe the **largest (worst) PDOP**? At what surface location do you see the **smallest (best) PDOP**?

ANSWER

```

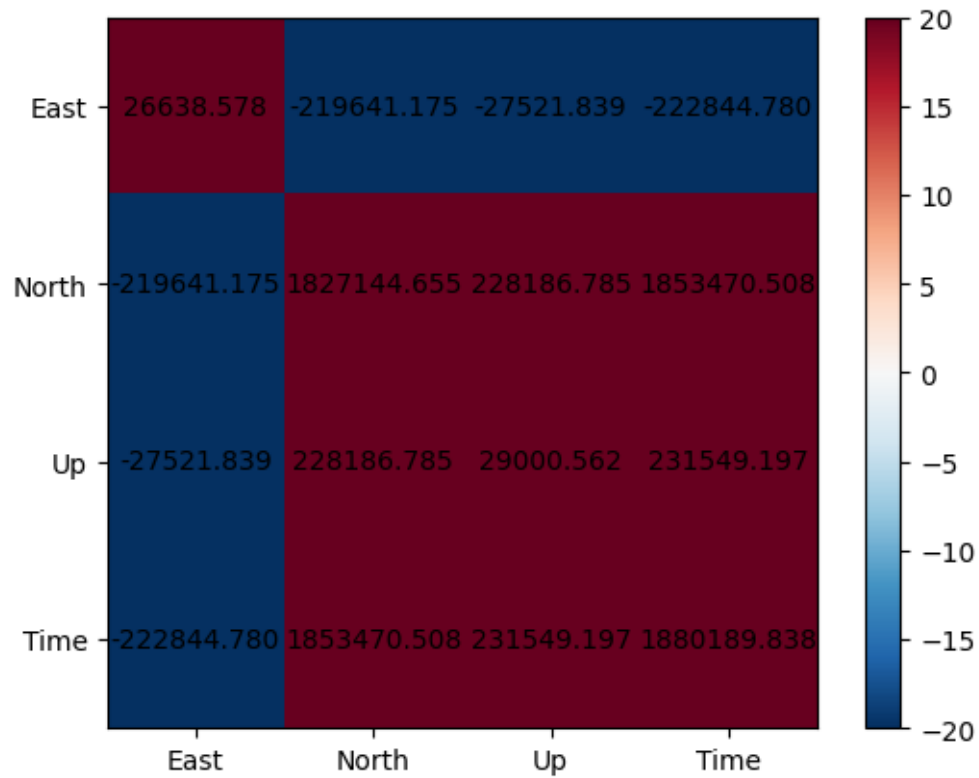
[18]: data_sh = pd.read_csv(f"{notebooks_dir}/data/hw2_p3_bonus_Shackleton.csv")
      data_mh = pd.read_csv(f"{notebooks_dir}/data/hw2_p3_bonus_MariusHills.csv")
      data_rb = pd.read_csv(f"{notebooks_dir}/data/hw2_p3_bonus_RimaBode.csv")
      data_gd = pd.read_csv(f"{notebooks_dir}/data/hw2_p3_bonus_GruithuisenDomes.csv")

      data_sh_az = data_sh['az_sv_deg']
      data_sh_el = data_sh['el_sv_deg']
      data_mh_az = data_mh['az_sv_deg']
      data_mh_el = data_mh['el_sv_deg']
      data_rb_az = data_rb['az_sv_deg']
      data_rb_el = data_rb['el_sv_deg']
      data_gd_az = data_gd['az_sv_deg']
      data_gd_el = data_gd['el_sv_deg']

[19]: H_sh = compute_dop_matrix(data_sh_az, data_sh_el, use_degree=True)
      plot_dop_matrix(H_sh, vmin=-20, vmax=20)
      print(f"PDOP: {compute_pdop(H_sh)}")

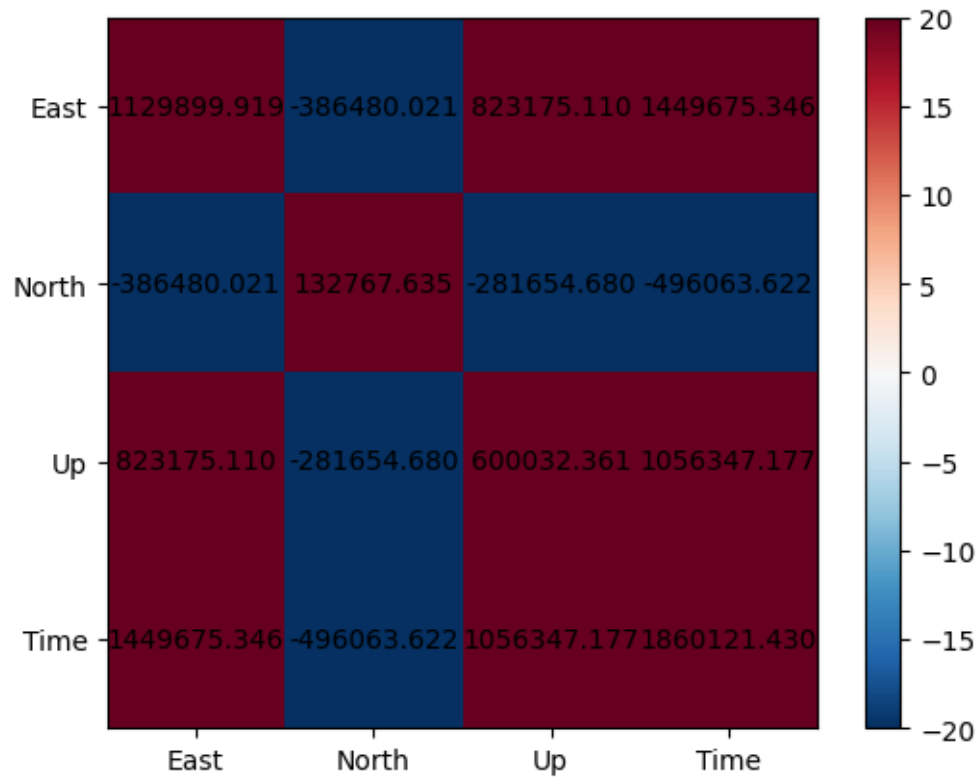
```

PDOP: 1372.145690066752



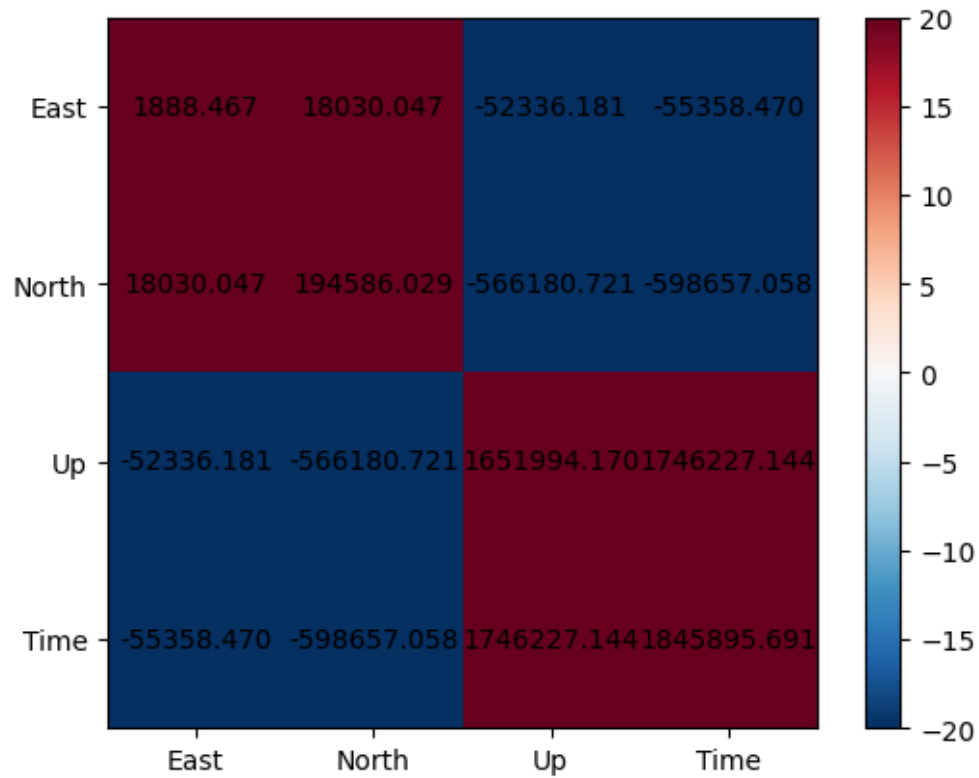
```
[20]: H_mh = compute_dop_matrix(data_mh_az, data_mh_el, use_degree=True)
      plot_dop_matrix(H_mh, vmin=-20, vmax=20)
      print(f"PDOP: {compute_pdop(H_mh)}")
```

PDOP: 1364.807647794641



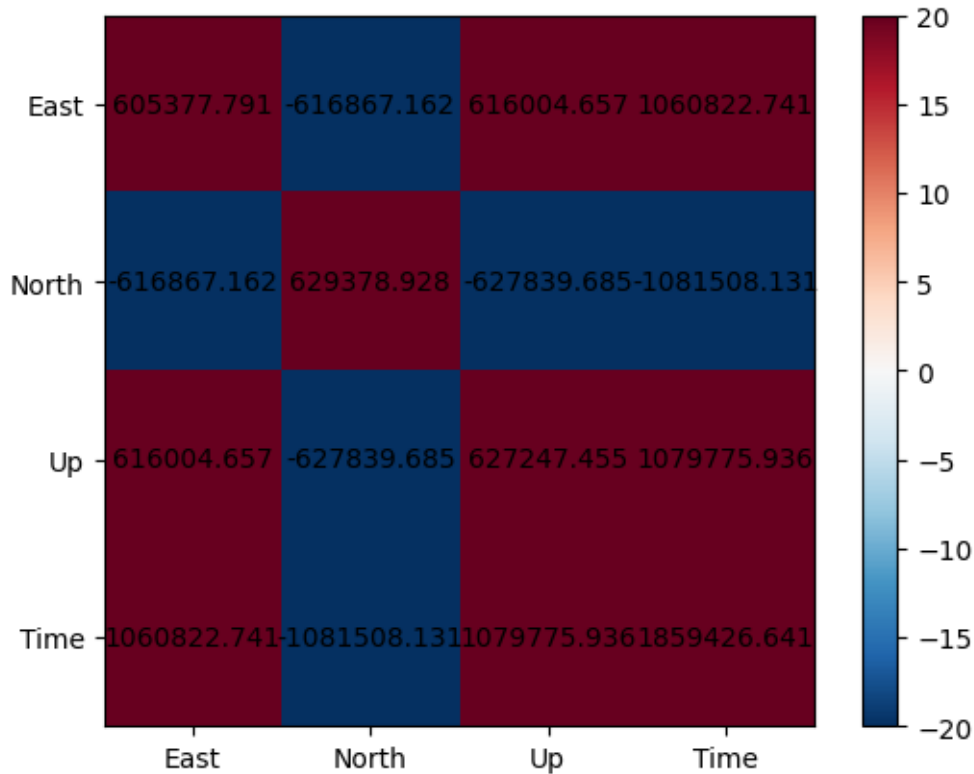
```
[21]: H_rb = compute_dop_matrix(data_rb_az, data_rb_el, use_degree=True)
      plot_dop_matrix(H_rb, vmin=-20, vmax=20)
      print(f"PDOP: {compute_pdop(H_rb)}")
```

PDOP: 1359.584004912472



```
[22]: H_gd = compute_dop_matrix(data_gd_az, data_gd_el, use_degree=True)
      plot_dop_matrix(H_gd, vmin=-20, vmax=20)
      print(f"PDOP: {compute_pdop(H_gd)}")
```

PDOP: 1364.5527376552623



They have about the same PDOP, but the locations with a more direct line-of-sight of the Earth have a slightly lower PDOP.

The smallest PDOP is 1359.6 for Rima Bode (with satellites almost directly overhead), and the largest is 1372.1 for Shackleton (with satellites clustered near the horizon).

3.8 Bonus [1 point] For each of the 4 locations, compute the **Time Dilution of Precision (TDOP)** values. How does it compare with PDOP? (Is it significantly larger, significantly smaller, or nearly the same?) Why do you think this happens?

ANSWER

```
[25]: tdop_sh = np.sqrt(H_sh[3,3])
      tdop_mh = np.sqrt(H_mh[3,3])
      tdop_rb = np.sqrt(H_rb[3,3])
      tdop_gd = np.sqrt(H_gd[3,3])

      print(f"TDOP (sh): {tdop_sh}")
      print(f"TDOP (mh): {tdop_mh}")
      print(f"TDOP (rb): {tdop_rb}")
      print(f"TDOP (gd): {tdop_gd}")
```

```
TDOP (sh): 1371.200144992796
TDOP (mh): 1363.8626874375789
```

TDOP (rb): 1358.6374392077305

TDOP (gd): 1363.607949784367

The PDOP and TDOP are nearly the same. Because the satellites are close together in the sky, the vectors between the receiver and each satellite are essentially the same. This causes the rows of the G matrix to be essentially the same, which makes the diagonals about the same. Doing the math for H , this causes all diagonals to be about the same as well, which is why the PDOP and TDOP are about the same (both calculated from diagonals of the H matrix).

3.9 Bonus [1 point] Consider navigating a user at **Shackleton** (which is located near the south pole). If you can add one more navigation satellite (a lunar GPS satellite!) anywhere in the sky to improve the PDOP (at this particular time epoch), where do you think would be a good location to add the new navigation satellite? Why do you think that would be a good location?

You do not have to compute the optimal location, nor compute the new DOP values. Also, You do not have to consider about the feasibility of the orbits.

ANSWER

A good location for the satellite would be on a polar with its orbital plane orthogonal to the moon's vector to Earth. Even though it is at low elevation, it would be a good location because there is no atmospheric or physical attenuation on the moon. It would also cause the satellites to be approximately in a line, but it would significantly improve one direction of error.

Self-care

Oftentimes, when we think of well-being, we might focus on the absence of illness or negative emotions. But, in addition to preventing ill health, we may further benefit by actively promoting flourishing in our lives. Reflect on this illuminating quote from Prof. Lynn Soots:

“Flourishing is the product of the pursuit and engagement of an authentic life that brings inner joy and happiness through meeting goals, being connected with life passions, and relishing in accomplishments through the peaks and valleys of life.”

What is 1 thing you have recently done (or 1 thing you will set an intention to do) which has helped you flourish in this moment of your life, whether it’s a “peak” or “valley”? Describe your thoughts in a few sentences. Alternatively, feel free to be creative in your form of expression, including but not limited to, a photo, video, or sketch.

ANSWER

Lately, I have been practicing with my band. We currently have a 6 song setlist with a mashup of Fall Out Boy’s [Sugar We’re Going Down](#) and 5 Seconds of Summer’s [cover of Teenage Dream](#). It’s a great break from the stress of school and career. I’ve also written several songs, which is very fun.

To be honest, a lot of stuff I’ve been doing to destress has been other fun engineering projects, like the 6-DOF sim on the SSI satellite team, or researching fun ideas for my projects in classes and in the NASA RASC-AL competition. Although that’s not technically “unplugging” from STEM, it is still something fun that I love to do.